

Your Code Is Slower Than Expected

Hidden Performance Traps

Swiss Python Summit 2026 – Michael Inden

OST
Ostschweizer
Fachhochschule

Speaker Intro Michael Inden

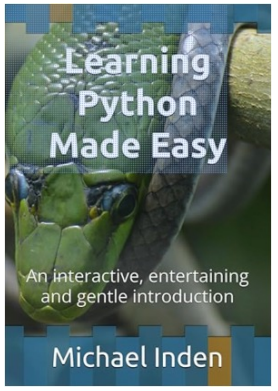


michael_inden@hotmail.com
michael.inden@ost.ch

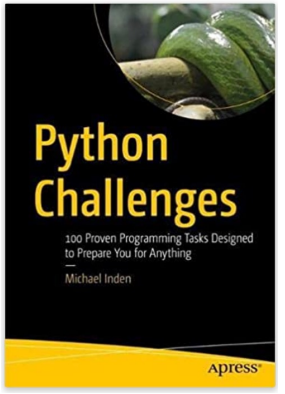
- Diploma Computer Science @ C.v.O. Uni Oldenburg
- ~8 ¼ Years SSE @ Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Years TPL, SA @ IVU Traffic Technologies AG in Aachen
- ~4 ¼ Years LSA / Trainer @ Zühlke Engineering AG in Zurich
- ~3 Years TL / CTO @ Direct Mail Informatics / ASMIQ in Zurich
- ~3 ½ Years Head of Development @ Adcubum in Zurich
- Since Apr 2026 Lecturer in Software Engineering @ OST in Rappi
- Independent Consultant, Conference Speaker and Trainer
- Author @ dpunkt.verlag and APress











OST

Agenda

AGENDA

DURATION	SECTION	KEY IDEA
~3 min	HOOK: "WHY IS THIS SLOW?"	Surprising performance example
~4 min	MENTAL MODEL: BIG O + DATA STRUCTURES	Performance begins at algorithm & data structures
~6 min	LISTS: INNOCENT-LOOKING TRAPS	+, append, comprehension, pop(0)
~5 min	MEMBERSHIP: LIST VS. SET/DICT	O(n) → O(1)
~4 min	UNDER THE HOOD	Why do these data structures behave this way?
~6 min	PERFORMANCE FIREWORKS	4–5 spectacular improvements
~2 min	TAKEAWAYS	Mental Model + Rules of Thumb

Think



We all remember situations where

- **Python makes it easy to write clean and expressive code ☺**
- **But it also makes it easy to write code that is unexpectedly slow ☹**

🎬 Introduction:
„Why is this slow?“



Motivation: Data Structures Matter

- This **every day code** looks **innocent** and **convenient:**

```
numbers = list(range(1_000_000))

def count_desired_numbers(desired_numbers, numbers):
    count = 0
    for num in desired_numbers:
        if num in numbers:
            count += 1
    return count
```

- **Is this fast or slow? Educated guesses?**
- What if we change just one line?



DEMO: `motivation.py`



OST

Key Takeaways

- Both versions are **correct**
- Both are **readable**
- Both **look harmless**
- **But their runtime behavior is completely different**
- The problem is **not Python syntax**
- The problem is the **mental model of the usage of data structures**

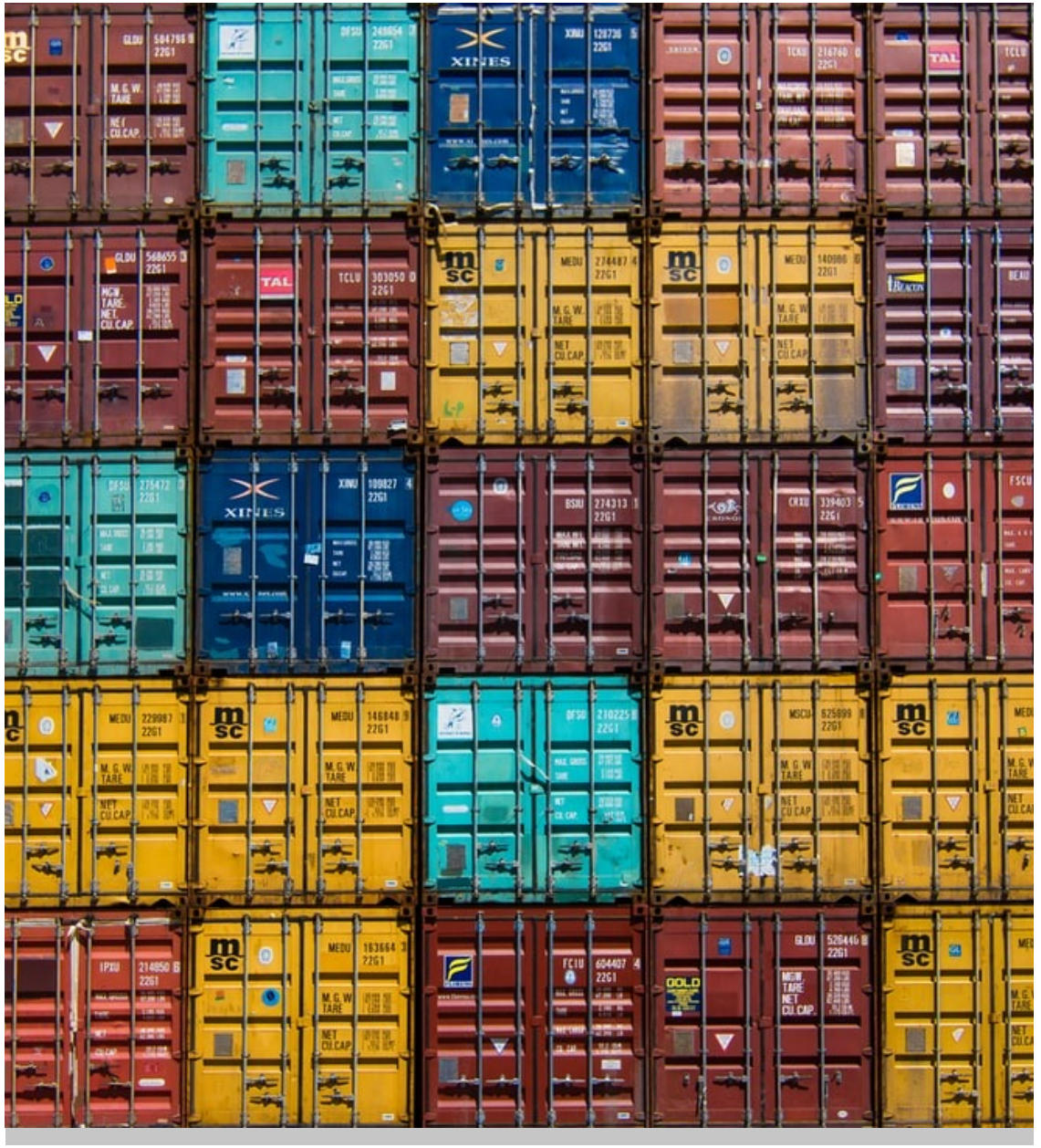
Highly dependent on system load, typically 6.000x – 10.000x

```
list: 17.9114s
set:  0.0022s
factor: 8012.9x
```

Performance doesn't start with optimizing code, but with choosing the right algorithms and data structures.

🧠 Mental Model

Big O



OST

🧠 **Mental Model**

Growth of Running Time When Input Data Grows

```
def find_value(values, target):
    for value in values:
        if value == target:
            return True
    return False
```

„What happens when the input gets 10× bigger?"

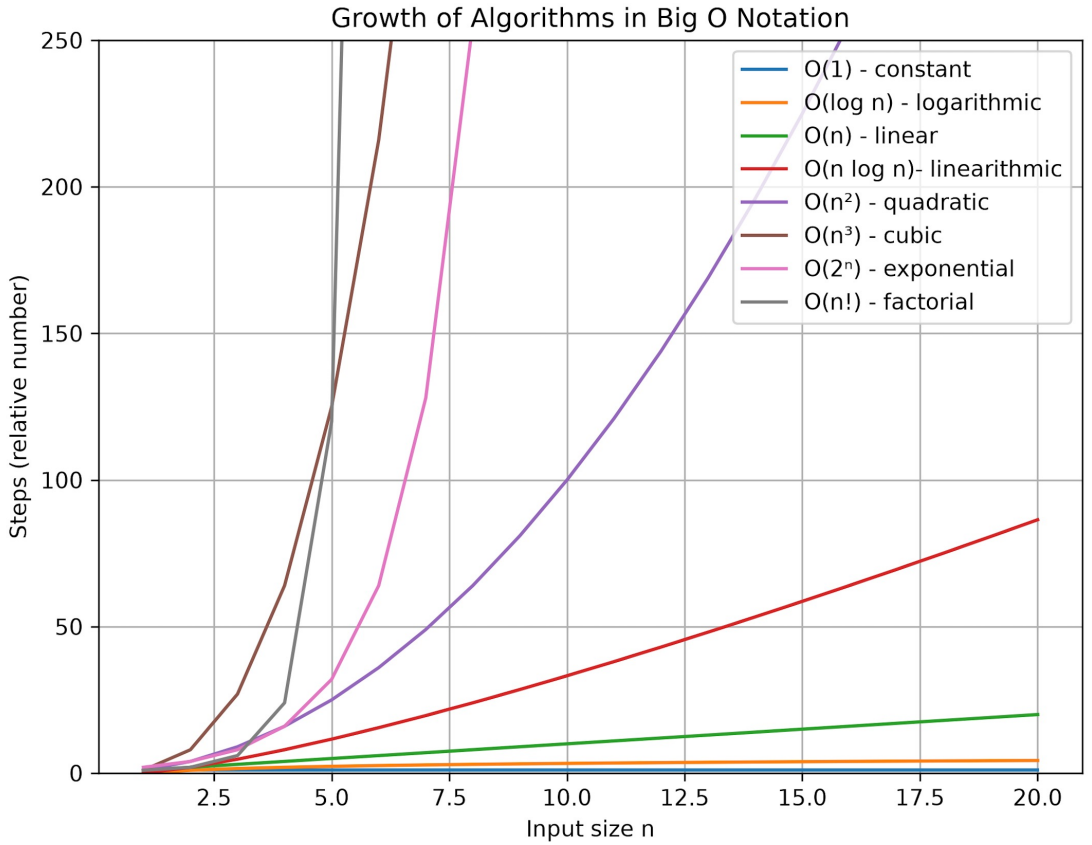
- **Linear Scaling O(n)**

  100 items → ~100 checks
  1'000 items → ~1'000 checks
  10'000 items → ~10'000 checks
  100'000 items → ~100'000 checks
  1'000'000 items → ~1'000'000 checks

Takeaway
Big O isn't about how fast something is.
It's about how the cost grows.

OST

Big O Isn’t About Concrete Runtime. It’s About the Grows!



Complexity	Name	Example
$O(1)$	constant	Accessing an array / list element by index
$O(\log n)$	logarithmic	Repeatedly halving the problem (e.g. “guess the number” with hints)
$O(n)$	linear	Iterating through a list once
$O(n \cdot \log n)$	linearithmic	Repeatedly dividing and combining
$O(n^2)$	quadratic	Two nested loops
$O(n^3)$	cubic	Three nested loops
$O(2^n)$	exponential	Trying all possibilities (e.g. determining all subsets)
$O(n!)$	factorial	Determining all permutations

„Remember this! In ~15 minutes we're going to turn some curves into flat lines.“

OST

📦 Lists

Innocent-looking traps



Four Ways Creating / Appending to a List

- **Different actions**

```
# A
result = result + [value]

# B
result.append(value)

# C
result += [value]

# D
result = [f(x) for x in values]
```

A: + **creates** a **new** list
- Existing elements have to be copied!

B: `append()` **modifies** the **existing** list

C: += **modifies** the **existing** list

D: List comprehensions are not magic, but they are often a **very efficient** and **idiomatic** approach.

Takeaway
Repeatedly copying data?

Seemingly Harmless Operation

- **Why is + different? It looks like we're just adding one element.**

```
result + [x]
```

- **But we're not adding an element — we're creating a new list.**

```
old list                    new list
[A][B][C]          →        [A][B][C][x]
```

↳ **copy everything**

DEMO: `four_ways_of_appending.py`

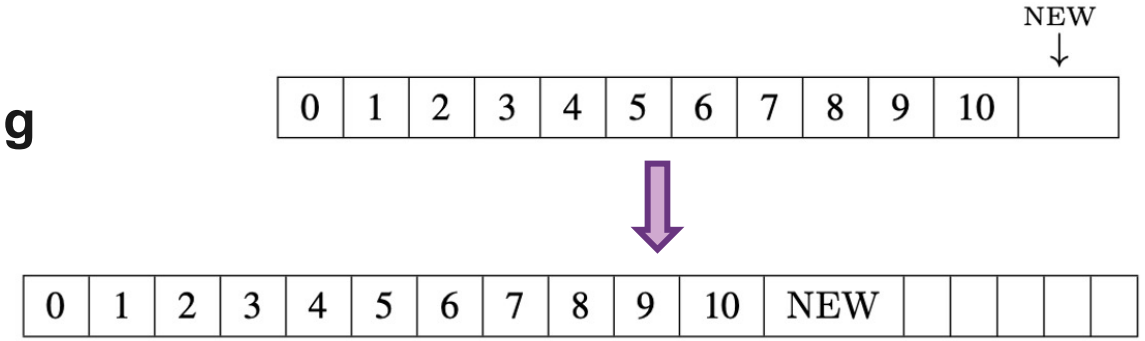


OST

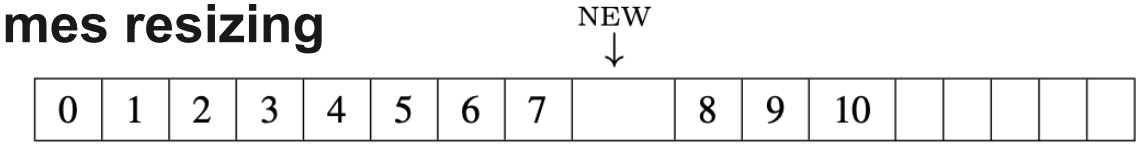
💣 `append()` / `append(mid)` / `pop(0)`

Actions causing elements to move:

- `append()` => sometimes resizing + copying
  => amortized `O(1)`



- `append(mid)` => always copying + sometimes resizing
  => `O(n)`



- `pop(0)` => always copying
  => `O(n)`

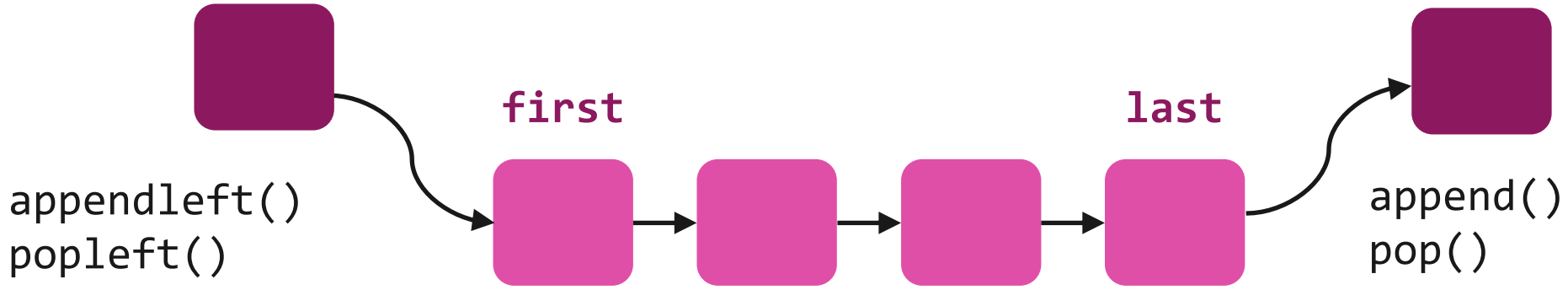
```
[A][B][C][D][E]
 ↓
pop(0)

[B][C][D][E][ ]
 ↑  ↑  ↑  ↑
move everything
```


OST

Solution: collections.deque

- **deque** = Doubled Ended Queue (fast insert/delete at front and back)




```
from collections import deque

queue = deque()
queue.append("LAST")
queue.appendleft("FIRST")

first = queue.popleft()
Last = queue.pop()
```

Takeaway
Choose your data structure based on the operations you need.

OST

🔍 Membership Test



OST

War Story: The Junior Removed Just One Line …

```
def check_membership(values, candidates):
    for candidate in candidates:
        if candidate in values:
            perform_action(candidate)
```

- **Original usage**:

```
values = set(orig_values)
check_membership(values, candidates)
```

`set` → **O(n)**

- **Junior «optimized» «unnecessary» code:**

```
# values = set(orig_values)
check_membership(orig_values, candidates)
```

`list` → **O(n × m)**

Same algorithm.
Same function.
Same data.
One line removed.

Recap and Enhancement

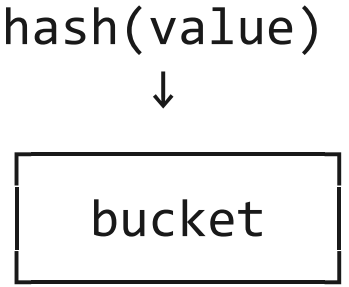
- **Membership test based on keyword «in»**

```
if value in my_list:
```



vs.

```
if value in my_set:
if key in my_dict:
```



Takeaway

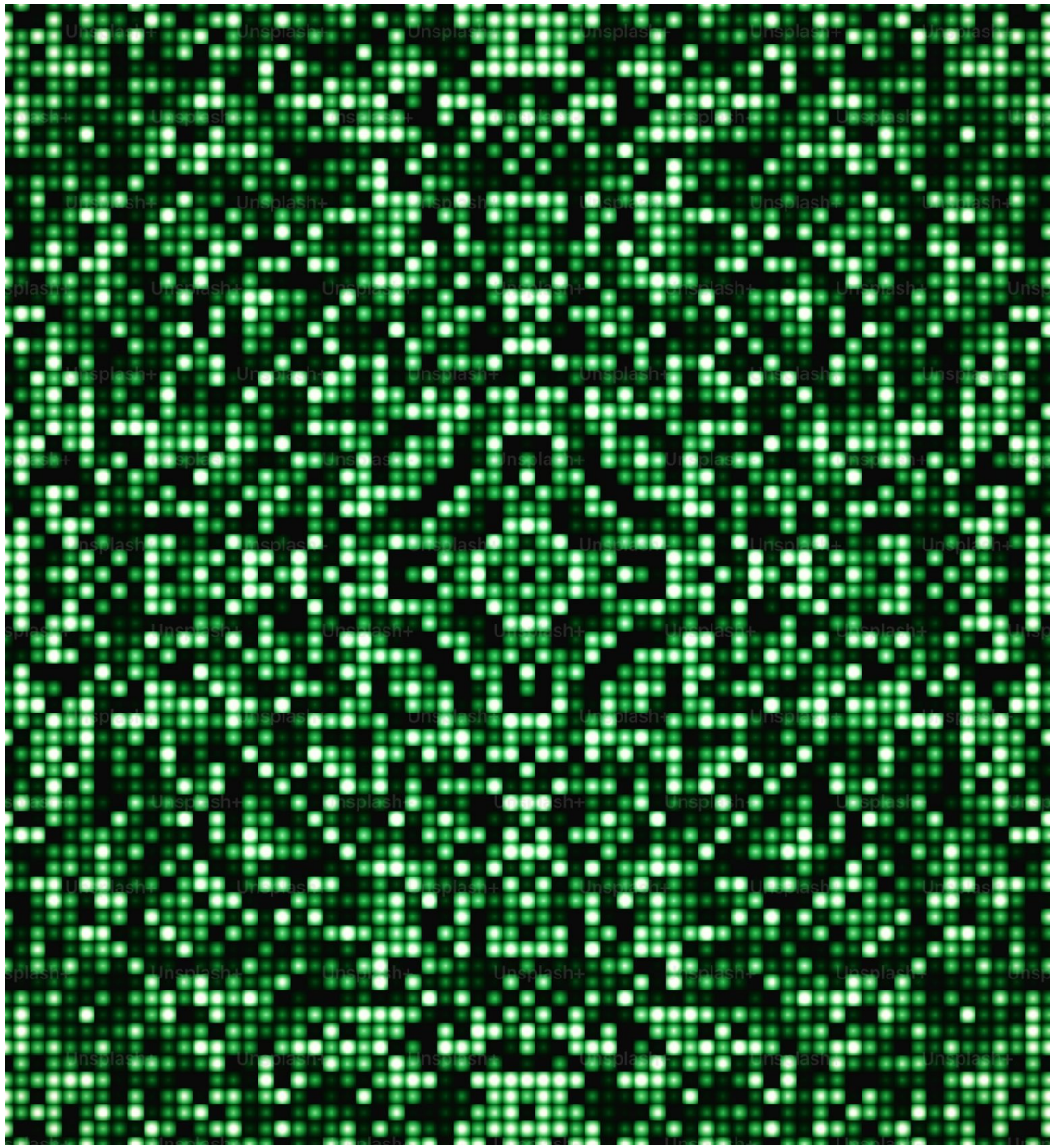
```
value in my_list # O(n)
value in my_set # O(1)*
```


OST

DEMO: `membership_test.py` / `membership_list_set.py`



🧪 What is actually happening under the hood?

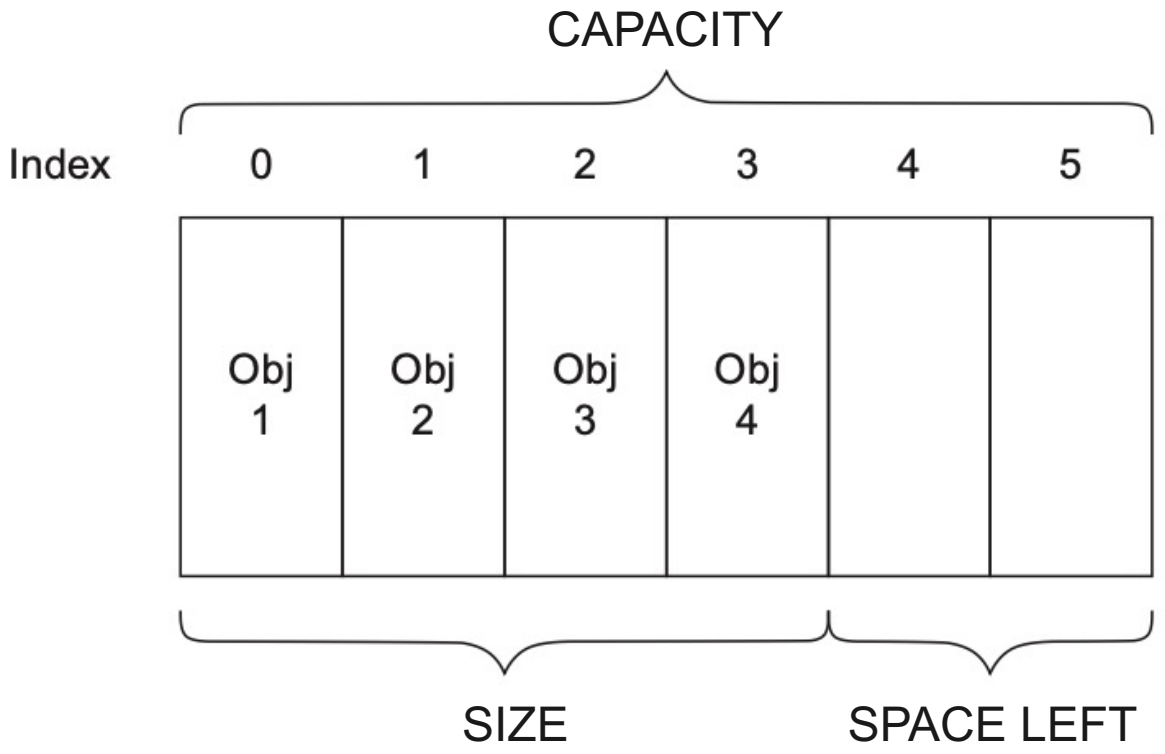


OST

🧪 **What is actually happening under the hood?**

Recap and Enhancement

- **`list` is conceptionally a dynamic array**
  - contiguous storage
  - index → direct access
  - resizing
  - shifting when inserting/removing at the beginning

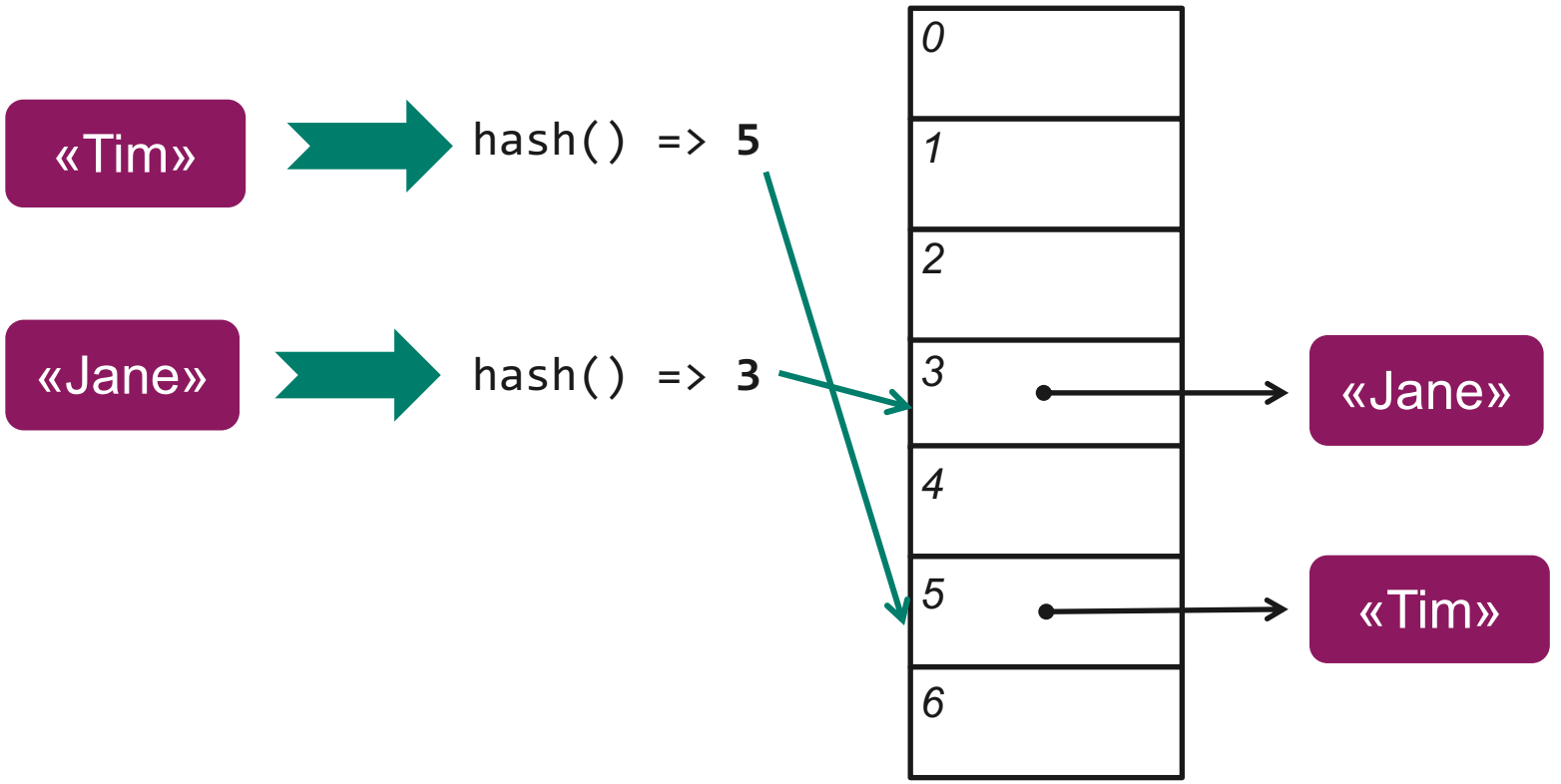


OST

🧪 What is actually happening under the hood?

Recap and Enhancement

- **`set` and `dict` are conceptionally look-up tables**
  - Hashing
  - Consume more memory
  - extremly fast membership and look-up operations



OST

🧪 **What is actually happening under the hood?**

Recap and Enhancement

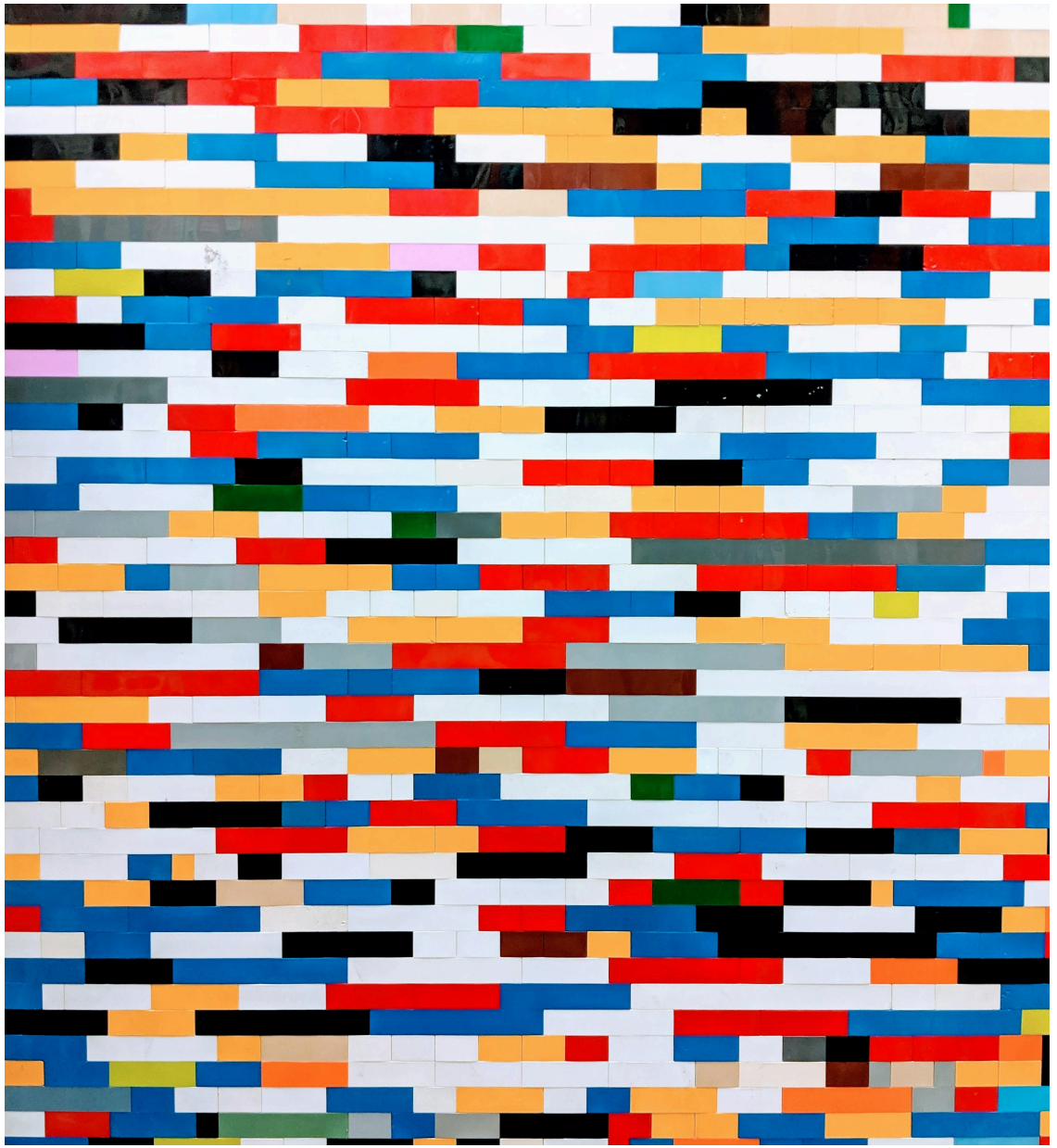
- **Keep in mind: O(1) doesn't mean "free".**
  - `set` and `dict` consume more memory (not all buckets occupied)
  - Hashing / collisions
  - Resizing and rehashing
  - Access overhead (`hash()`)

„Set is NOT always faster than list."

- **Big O helps you reason about scaling, not about every single nanosecond.**

🔥 Performance Fireworks

„Let's turn some bad curves into better curves or even flat lines.“



OST

Data Structure and Operations Pitfall: O(n²) → O(n)

- **We already know this bad pattern:**

```
result = []
for item in data:
    result = result + [transform(item)]
```

Looks like O(n)! But is actually O(n²)!

```
1 element → copy 1
2 elements → copy 2
...
n elements → copy n
=> 1 + 2 + 3 + ... + n = O(n²)
```

- **The algorithm isn't the problem — misusing the data structure causes unnecessary copying.**

```
result = [transform(item) for item in data]
```

Takeaway

Know the specialities of data structures and their methods to avoid pitfalls!

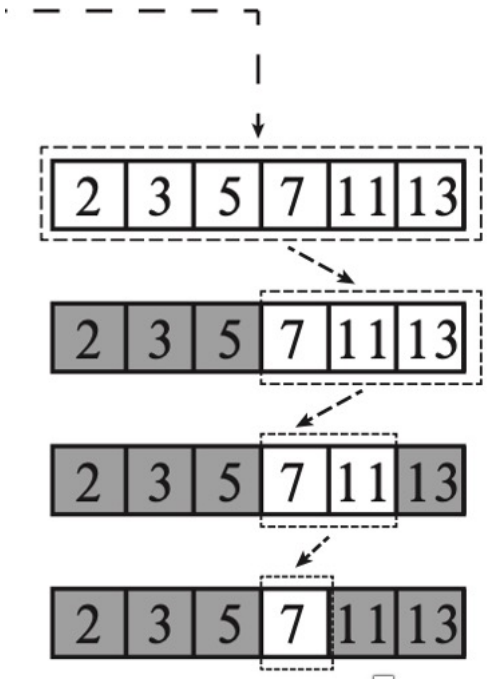
Algorithm: Linear vs. Binary Search: O(n) → O(log n)

- **Linear Search, if no info about data**

```
def linear_search(values, target):
    for idx, val in enumerate(values):
        if val == target:
            return idx
    return -1
```



- **Binary Search, if you know data is sorted**
  - Always skip half of the values until you reach target (or fail)



Takeaway

Sometimes the biggest optimization is choosing a different algorithm. Challenge the algorithm before you optimize the implementation.

Algorithm: Find all Primes ≤ n: $O(n^2) \rightarrow O(n \sqrt{n}) = O(n^{1.5})$

- **Step 1: Just start coding**

```
def is_prime(n):
    if n < 2:
        return False

    for d in range(2, n):
        if n % d == 0:
            return False
    return True


def naive_calc_primes(n):
    return [n for n in range(2, n + 1)
            if is_prime(n)]
```

Skip irrelevant upper half → n/2

```
def is_prime(n):
    # …
    for d in range(2, n // 2 + 1):
    # …
```

Check only up to √n

```
def is_prime(n):
    # …
    d = 2
    while d * d <= n:
    # …
```

"The code fis aster—but not the algorithm."

"We only need to check up to √n

Takeaway

Look for better fitting (mathematical) model.

OST

Algorithm: Prime Numbers: O(n √n) → O(n log log n)

- **Shift in thinking:** “Why testing numbers repeatedly & individually? Try eliminate in one run.”
- **Sieve of Erastothenes**

$$2, 3, \cancel{4}, 5, \cancel{6}, 7, \cancel{8}, \cancel{9}, \cancel{10}, 11, \cancel{12}, 13, \cancel{14}, \cancel{15}$$

```python
def calc_primes_sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False

    p = 2
    while p * p <= n:
        if is_prime[p]:
            for multiple in range(p * p, n + 1, p):
                is_prime[multiple] = False
        p += 1

    return [i for i in range(2, n + 1)
            if is_prime[i]]
```

Takeaway
The Sieve of Eratosthenes is ***almost linear*** in practice: $O(n \log \log n)$.

OST

Algorithm: Calculate Sum: Loop vs. Formula: O(n) → O(1)

- We want to calculate the sum of first n integers: $\sum_{i=0}^{n-1} i$

- First “good” implementation — but is the loop really necessary?

```
sum = 0
for i in range(n):
    sum += i
```

„Don't optimize the loop. Remove the loop.“

- **Algorithm is problematic** — replace iteration with pure math (Gauss): $\text{sum} = \frac{n(n-1)}{2}$

```
sum = n * (n - 1) // 2
```

Takeaway
Can we remove or changed the algorithm entirely?
Change the algorithm before you optimize the implementation.

Caching Tricks: Recursion & Memoization $O(2^n) \rightarrow O(n)$

- **Naive Fibonacci:** simple recursion, elegant but **painfully slooooooooow**.
  Looks innocent — but recomputes the same values exponentially often.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

- **Fix it using memoization (just one line if using built-in decorator):**

```
@lru_cache
def fib_optimized(n):
    if n <= 1:
        return n
    return (fib_optimized(n - 1) +
            fib_optimized(n - 2))
```

Takeaway
Same algorithm, but using caching, dramatically faster.

Recursion, Memoization, Formula: O(2ⁿ) → O(n) → O(1)

- There is even mathematical formula for Fibonacci that are significantly faster.
- **Binet's formula*** $F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$ where $\varphi = \frac{1+\sqrt{5}}{2}, \quad \psi = \frac{1-\sqrt{5}}{2}$

```
def fibonacci(n):
    phi = (1 + math.sqrt(5)) / 2
    psi = (1 - math.sqrt(5)) / 2
    return round((phi**n - psi**n) / math.sqrt(5))
```

Takeaway:
Change the algorithm before you optimize the implementation.

🔥🔥🔥 DEMO: `firework_1/2/3/4/5_*.py` 🔥🔥🔥



- 🔥 1 list → comprehension
- 🔥 2 linear → binary search
- 🔥 3 prime numbers → sieve
- 🔥 4 loop → formula
- 🔥 5 recursion → memoization

OST

🎯 Conclusion



OST

Your Performance Cheat Sheet

When your code is slow, ask yourself:

1. What operation?
2. On what data structure?
3. How often?

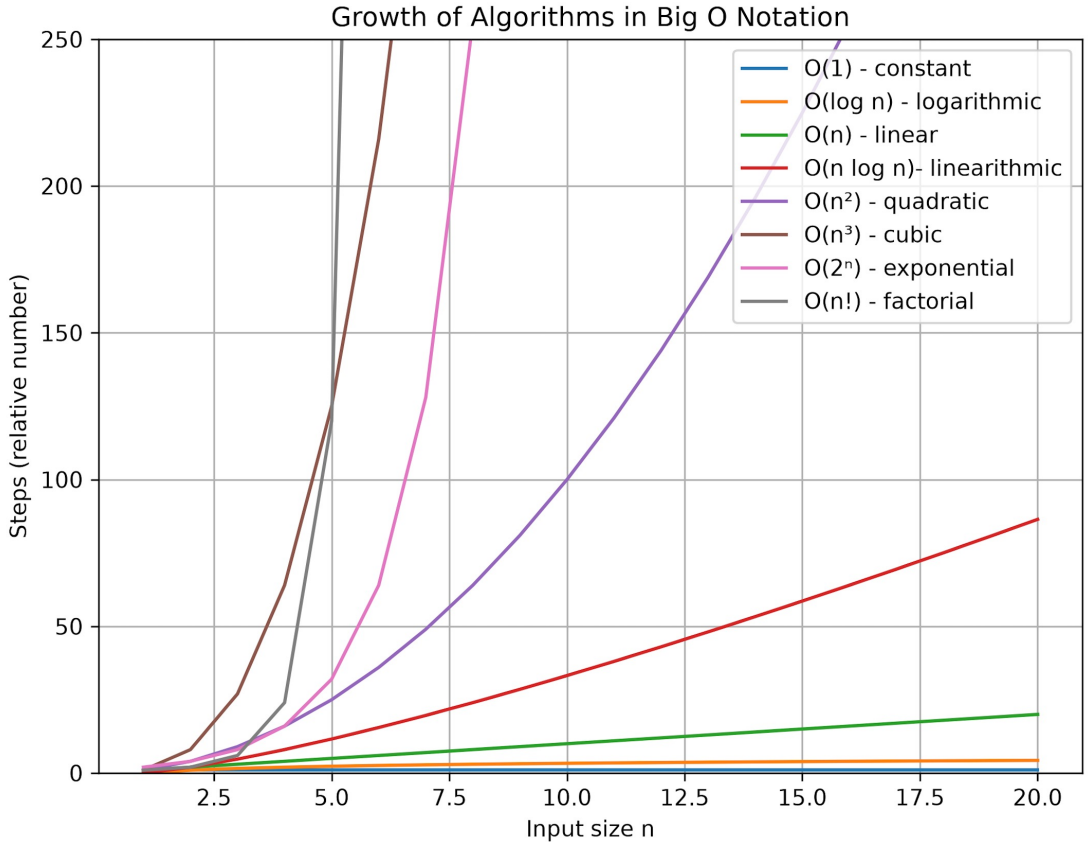
Operation	`list`	`set` / `dict`
Membership “`x in values`”	O(n)	O(1)*
`append()` at end / `add()`	O(1)*	O(1)*
`pop(0)` (remove first)	O(n)	—
Access by index	O(1)	—

🎯 Conclusion

Your Performance Mental Model

1. **What is the running time of my algorithm?**

   `O(1)` < `O(log n)` < `O(n)` < `O(n log n)` < `O(n²)` < `O(2ⁿ)`



2. **What is my data structure?**

   - `list` → ordered sequence
   - `set` → membership
   - `dict` → lookup by key
   - `deque` → queue / fast at both ends

3. **What happens as my data grows?**

   - **Key Takeaway**: Don't optimize what happens once. Optimize what happens very often.

OST